

Test Plan

Test Strategy

The test strategy used will be an analytical strategy, relying on both risk-based and requirements-based testing.

Automated Tools

Various types of automated tools will be used throughout the development cycle.

Firstly, a code editor such as VSCode will be used for static analysis, in order to catch “bugs” and “smells” in the development phase.

Secondly, various code sanitizers will be used throughout development, debugging, and testing, namely:

- ThreadSan, in order to catch deadlocks and data races,
- UBSan in order to catch undefined behaviour, and
- LeakSan in order to check if memory is allocated but not freed.

Lastly, fuzzers will be used to provide invalid, or random data to the modules, in order to uncover any unexpected issues. A fuzzer such as Atheris will be used to test the Python visualisation and graphing, and a fuzzer such as the one found at github.com/CodeIntelligenceTesting/jazzer will be used to test the Java simulation.

Unit Testing

Sonali will be responsible for ensuring test coverage, using coverage testing software such as Clover or Cobertura for the Java sections, and Coverage.py for the Python sections. In the event that the coverage is insufficient, the test cases will be reviewed, and expanded upon. The main unit testing frameworks to be used are JUnit and PyUnit (unittest).

Test Cases

Test Case 1: Input Validation

Description: Check that the validation of the input file works as expected.

Input: Correctly formatted csv file containing data of the correct type (float).

Expected Behaviour: The module reads in the contents of the file and checks that each value is a float.

Expected Output: Boolean “True”.

Test Case 2: Calculation Accuracy

Description: Check that the equations used in the simulation produce the correct values.

Input: Float values from the csv input file.

Expected Behaviour: Each equation outputs the correct value for that equation.

Expected Output: Mathematically correct value.

Test Case 3: Simulation Visualisation

Description: Ensure the data generated in the output file by the simulation module is visualised correctly.

Input: Data from a time step contained in the simulation output file.

Expected Behaviour: The data for a time step is used to generate a visual representation of the colony.

Expected Output: The colony visualisation correlates to the data contained in the simulation output file, including updated bacteria size, position, and orientation.

Test Case 4: Graph Generation

Description: Verify that the graphs generated are accurate representations of the simulation data saved at regular intervals.

Input: Output file containing simulation data saved at regular intervals.

Expected Behaviour: Module generates accurate and readable graphs based on the output file.

Expected Output: Accurate and readable graphs based on corresponding data.

Test Case 5: Error Handling

Description: Check that various error handling steps are functional and don't cause the module to crash.

Input: Invalid input file.

Expected Behaviour: Module prompts user to input a file with correct structure, without causing an error.

Expected Output: Message reminding user of acceptable input types ie csv file.

Additional Actions

Additional actions to be take during the testing phase include:

Mocking - making use of dummy methods and classes is especially useful when testing OOP projects.

Integration testing - will be useful when testing the interactions between the simulation, visualisation, and graphing modules.

User acceptance - external individuals will test run the program to ensure it is intuitive for individuals unfamiliar with the source code.